TRƯỜNG ĐẠI HỌC SÀI GÒN

KHOA CÔNG NGHỆ THÔNG TIN

GENERATIVE AI

Ver 1.1

Bài tập lớn:

SmartDoc AI - Intelligent Document Q&A System

Môn học:	Open source software development
Học kỳ:	Spring 2026
Thời hạn:	April 19, 2026

TP. HỒ CHÍ MINH, NĂM 2026

Mục lục

LỜI MỞ ĐẦU **4**

1 GIỚI THIỆU **5**
1.1 Bối cảnh và động lực 5
1.2 Mục tiêu dự án 5
1.3 Phạm vi dự án 5

2 CƠ SỞ LÝ THUYẾT **6**
2.1 RAG (Retrieval-Augmented Generation) 6
2.1.1 Khái niệm 6
2.1.2 Kiến trúc RAG 6
2.2 Embedding và Vector Database 6
2.2.1 Text Embedding 6
2.2.2 FAISS (Facebook AI Similarity Search) 6
2.3 Large Language Models (LLMs) 7
2.3.1 Qwen2.5 7
2.3.2 Ollama 7
2.4 LangChain Framework 7

3 THIẾT KẾ HỆ THỐNG **8**
3.1 Kiến trúc tổng quan 8
3.2 Data Flow (Luồng dữ liệu) 8
3.2.1 Document Processing Flow 8
3.2.2 Query Processing Flow 9
3.3 Chi tiết các thành phần 9
3.3.1 Document Loader 9
3.3.2 Text Splitter 9
3.3.3 Embedding Model 10
3.3.4 Vector Store 10
3.3.5 LLM Integration 10
3.4 Prompt Engineering 11

4 TRIỂN KHAI **12**
4.1 Công nghệ sử dụng 12
4.1.1 Frontend 12
4.1.2 Backend & AI 12
4.1.3 Document Processing 12
4.1.4 Python Libraries 12
4.2 Cài đặt môi trường 12
4.2.1 Yêu cầu hệ thống 12
4.2.2 Các bước cài đặt 13
4.3 Cấu trúc thư mục 13
4.4 Cấu hình chi tiết 13
4.4.1 Embedding Configuration 13
4.4.2 Retriever Configuration 14
4.4.3 LLM Configuration 14

5 GIAO DIỆN NGƯỜI DÙNG 15
5.1 Thiết kế UI/UX . . . 15
5.1.1 Color Palette . . . 15
5.1.2 Layout Structure . . . 15
5.2 User Flow . . . 15
5.3 Features . . . 16
5.3.1 File Upload . . . 16
5.3.2 Question Answering . . . 16
5.3.3 Error Handling . . . 16

6 KẾT QUẢ VÀ ĐÁNH GIÁ 17
6.1 Kết quả đạt được . . . 17
6.1.1 Chức năng hoàn thành . . . 17
6.1.2 Performance Metrics . . . 17
6.2 Testing . . . 17
6.2.1 Test Cases . . . 17
6.3 Ưu điểm . . . 18
6.4 Hạn chế . . . 18
6.5 So sánh với các giải pháp khác . . . 18

7 HƯỚNG DẪN SỬ DỤNG 19
7.1 Hướng dẫn cho người dùng cuối . . . 19
7.1.1 Khởi động ứng dụng . . . 19
7.1.2 Upload tài liệu . . . 19
7.1.3 Đặt câu hỏi . . . 19
7.1.4 Tips sử dụng . . . 19
7.2 Hướng dẫn cho developers . . . 20
7.2.1 Customize embedding model . . . 20
7.2.2 Adjust chunk parameters . . . 20
7.2.3 Thay đổi LLM model . . . 20
7.2.4 Modify retrieval parameters . . . 20
7.2.5 Add logging . . . 21

8 YÊU CẦU PHÁT TRIỂN PROJECT 22
8.1 Giới thiệu . . . 22
8.2 Các yêu cầu phát triển . . . 22
8.2.1 Câu hỏi 1: Thêm hỗ trợ file DOCX . . . 22
8.2.2 Câu hỏi 2: Lưu trữ lịch sử hội thoại . . . 22
8.2.3 Câu hỏi 3: Thêm nút xóa lịch sử . . . 22
8.2.4 Câu hỏi 4: Cải thiện chunk strategy . . . 22
8.2.5 Câu hỏi 5: Thêm citation/source tracking . . . 23
8.2.6 Câu hỏi 6: Implement Conversational RAG . . . 23
8.2.7 Câu hỏi 7: Thêm hybrid search . . . 23
8.2.8 Câu hỏi 8: Multi-document RAG với metadata filtering . . . 23
8.2.9 Câu hỏi 9: Implement Re-ranking với Cross-Encoder . . . 23
8.2.10 Câu hỏi 10: Advanced RAG với Self-RAG . . . 24
8.3 Hướng dẫn đánh giá . . . 24

9 KẾT LUẬN **25**
9.1 Tổng kết 25

LỜI MỞ ĐẦU

Trong kỷ nguyên số hiện nay, việc xử lý và tìm kiếm thông tin từ các tài liệu điện tử đã trở thành một nhu cầu thiết yếu. Với sự phát triển mạnh mẽ của Trí tuệ nhân tạo (AI) và Xử lý ngôn ngữ tự nhiên (NLP), các hệ thống RAG (Retrieval-Augmented Generation) đã nổi lên như một giải pháp hiệu quả cho bài toán này.

Dự án này được thực hiện nhằm xây dựng một hệ thống RAG hoàn chỉnh, cho phép người dùng tải lên tài liệu PDF và đặt câu hỏi về nội dung tài liệu. Hệ thống sử dụng mô hình Qwen2.5:7b thông qua Ollama, kết hợp với công nghệ embedding đa ngôn ngữ và vector database để cung cấp câu trả lời chính xác cho cả tiếng Việt và tiếng Anh.

Báo cáo này trình bày chi tiết về kiến trúc hệ thống, các công nghệ được sử dụng, quy trình triển khai và kết quả đạt được.

1 GIỚI THIỆU

1.1 Bối cảnh và động lực

Với khối lượng thông tin ngày càng tăng trong các tài liệu số, việc tìm kiếm và trích xuất thông tin một cách hiệu quả đã trở thành thách thức lớn. Các phương pháp truyền thống như tìm kiếm từ khóa thường không đủ để hiểu ngữ cảnh và ý nghĩa sâu xa của câu hỏi.

Hệ thống RAG (Retrieval-Augmented Generation) kết hợp ưu điểm của:

- **Information Retrieval**: Tìm kiếm thông tin liên quan từ cơ sở dữ liệu
- **Natural Language Generation**: Sinh ra câu trả lời tự nhiên dựa trên thông tin đã tìm được

1.2 Mục tiêu dự án

Dự án đặt ra các mục tiêu cụ thể sau:

1. Xây dựng giao diện web thân thiện cho phép người dùng tải lên tài liệu PDF
2. Tích hợp công nghệ embedding để chuyển đổi văn bản thành vector
3. Triển khai vector database (FAISS) để lưu trữ và tìm kiếm hiệu quả
4. Tích hợp mô hình ngôn ngữ lớn Qwen2.5:7b được tối ưu cho tiếng Việt
5. Xây dựng pipeline RAG hoàn chỉnh từ document loading đến answer generation
6. Tối ưu hóa hiệu suất và độ chính xác của câu trả lời

1.3 Phạm vi dự án

Dự án tập trung vào:

- Xử lý tài liệu PDF đa ngôn ngữ
- Hỗ trợ xuất sắc cho tiếng Việt và 50+ ngôn ngữ khác
- Tự động phát hiện ngôn ngữ và trả lời phù hợp
- Chạy local trên máy tính cá nhân
- Sử dụng mô hình open-source và miễn phí

2 CƠ SỞ LÝ THUYẾT

2.1 RAG (Retrieval-Augmented Generation)

2.1.1 Khái niệm

RAG là một kỹ thuật trong NLP kết hợp hai thành phần:

1. **Retrieval (Truy xuất)**: Tìm kiếm thông tin liên quan từ cơ sở dữ liệu
2. **Generation (Sinh)**: Sử dụng LLM để sinh ra câu trả lời dựa trên thông tin đã truy xuất

2.1.2 Kiến trúc RAG

Kiến trúc RAG cơ bản bao gồm:

$$\text{RAG}(x) = \text{Generator}(x, \text{Retriever}(x)) \tag{1}$$

Trong đó:

- x: Câu hỏi đầu vào
- Retriever(x): Hàm truy xuất các tài liệu liên quan
- Generator: Mô hình sinh câu trả lời

2.2 Embedding và Vector Database

2.2.1 Text Embedding

Text embedding là quá trình chuyển đổi văn bản thành vector số có số chiều cố định:

$$E : \text{Text} \to \mathbb{R}^d \tag{2}$$

Trong đó d là số chiều của vector (thường là 384, 512, 768, v.v.)
Đặc điểm quan trọng:

- Văn bản có nghĩa tương tự sẽ có vector gần nhau trong không gian d chiều
- Khoảng cách được tính bằng cosine similarity hoặc Euclidean distance

2.2.2 FAISS (Facebook AI Similarity Search)

FAISS là thư viện hiệu quả cho similarity search và clustering của dense vectors:
Ưu điểm:

- Tìm kiếm nhanh với hàng triệu vector
- Hỗ trợ GPU acceleration
- Memory-efficient indexing

2.3 Large Language Models (LLMs)

2.3.1 Qwen2.5

Qwen2.5 là mô hình ngôn ngữ lớn phát triển bởi Alibaba Cloud:

Đặc điểm:

- Kiến trúc transformer-based thế hệ mới
- Hỗ trợ context dài lên đến 128K tokens
- Tối ưu xuất sắc cho tiếng Việt và đa ngôn ngữ
- Phiên bản 7B parameters cân bằng giữa hiệu suất và chất lượng
- Được train trên corpus đa ngôn ngữ bao gồm tiếng Việt
- Performance vượt trội so với Llama3 cho tiếng Việt

2.3.2 Ollama

Ollama là framework để chạy LLMs locally:

Tính năng:

- API đơn giản, dễ sử dụng
- Hỗ trợ nhiều mô hình khác nhau
- Tối ưu performance cho local machine
- Không cần internet sau khi tải model

2.4 LangChain Framework

LangChain là framework Python để xây dựng ứng dụng với LLMs:

Thành phần chính:

- **Document Loaders**: Tải và parse các định dạng tài liệu
- **Text Splitters**: Chia văn bản thành chunks nhỏ
- **Embeddings**: Chuyển đổi text thành vectors
- **Vector Stores**: Lưu trữ và tìm kiếm vectors
- **Retrievers**: Truy xuất thông tin liên quan
- **Chains**: Kết nối các thành phần lại với nhau

3 THIẾT KẾ HỆ THỐNG

3.1 Kiến trúc tổng quan

Hệ thống được thiết kế theo mô hình multi-layer architecture:

Presentation Layer (Streamlit)

- File Upload Interface
- Question Input Interface
- Answer Display

Application Layer (LangChain)

- Document Processing Pipeline
- RAG Chain Management
- Prompt Engineering

Data Layer

- FAISS Vector Store
- Multilingual MPNet Embeddings (768-dimensional)
- PDF Document Storage

Model Layer (Ollama)

- Qwen2.5:7b Model
- LLM Inference Engine

Hình 1: Kiến trúc hệ thống RAG

3.2 Data Flow (Luồng dữ liệu)

3.2.1 Document Processing Flow

1. **Upload**: Người dùng upload PDF file
2. **Loading**: PDFPlumberLoader đọc nội dung PDF
3. **Splitting**: RecursiveCharacterTextSplitter chia thành chunks
4. **Embedding**: Multilingual MPNet (768-dim) chuyển chunks thành vectors
5. **Indexing**: FAISS lưu trữ vectors vào index

3.2.2 Query Processing Flow

1. **Query Input**: Người dùng nhập câu hỏi
2. **Query Embedding**: Chuyển câu hỏi thành vector
3. **Similarity Search**: FAISS tìm top-k chunks liên quan nhất
4. **Context Building**: Kết hợp các chunks thành context
5. **Prompt Construction**: Tạo prompt với context và question
6. **LLM Inference**: Qwen2.5:7b sinh câu trả lời
7. **Response Display**: Hiển thị kết quả cho người dùng

3.3 Chi tiết các thành phần

3.3.1 Document Loader

```python
from langchain_community.document_loaders import PDFPlumberLoader

# Load PDF document
loader = PDFPlumberLoader("document.pdf")
docs = loader.load()
```

Listing 1: Document Loading với PDFPlumber

Đặc điểm PDFPlumberLoader:

- Trích xuất text chính xác, bao gồm tables
- Preserve layout information
- Xử lý được các PDF phức tạp

3.3.2 Text Splitter

```python
from langchain_text_splitters import RecursiveCharacterTextSplitter

text_splitter = RecursiveCharacterTextSplitter(
    chunk_size=1000,       # Length of each chunk
    chunk_overlap=100      # Overlap between chunks
)
documents = text_splitter.split_documents(docs)
```

Listing 2: Text Splitting

Tham số quan trọng:

- `chunk_size=1000`: Mỗi chunk tối đa 1000 ký tự
- `chunk_overlap=100`: 100 ký tự trùng lặp giữa các chunks liên tiếp
- Overlap giúp đảm bảo ngữ cảnh không bị mất ở ranh giới

3.3.3 Embedding Model

```python
from langchain_community.embeddings import HuggingFaceEmbeddings

# Using multilingual MPNet for Vietnamese support
embedder = HuggingFaceEmbeddings(
    model_name="sentence-transformers/paraphrase-multilingual-mpnet-base-v2",
    model_kwargs={'device': 'cpu'},
    encode_kwargs={'normalize_embeddings': True}
)
# Output dimension: 768
# Supports: Vietnamese, English, Chinese, and 50+ languages
```

Listing 3: HuggingFace Embeddings

Model được sử dụng: `paraphrase-multilingual-mpnet-base-v2`

- 768-dimensional embeddings (chất lượng cao)
- Hỗ trợ 50+ ngôn ngữ bao gồm tiếng Việt
- MPNet architecture - state-of-the-art cho multilingual
- Excellent quality cho semantic search tiếng Việt
- Được train trên corpus tiếng Việt lớn

3.3.4 Vector Store

```python
from langchain_community.vectorstores import FAISS

# Create vector store from documents
vector = FAISS.from_documents(documents, embedder)

# Create retriever
retriever = vector.as_retriever(
    search_type="similarity",
    search_kwargs={"k": 3}  # Retrieve top 3 results
)
```

Listing 4: FAISS Vector Store

3.3.5 LLM Integration

```python
from langchain_community.llms import Ollama

# Using Qwen2.5:7b for best Vietnamese support
llm = Ollama(model="qwen2.5:7b")
```

Listing 5: Ollama LLM

3.4 Prompt Engineering

Prompt được thiết kế để tối ưu hóa chất lượng câu trả lời:

```
# Auto-detect language and respond accordingly
vietnamese_chars = 'aaaaeeeiooouuuuyyyyd'
is_vietnamese = any(char in user_input.lower()
                    for char in vietnamese_chars)

if is_vietnamese:
    prompt_text = """Su dung ngu canh sau day de tra loi cau hoi.
Neu ban khong biet, chi can noi la ban khong biet.
Tra loi ngan gon (3-4 cau) BAT BUOC bang tieng Viet.

Ngu canh: {context}

Cau hoi: {user_input}

Tra loi:"""
else:
    prompt_text = """Use the following context to answer the question.
If you don't know the answer, just say you don't know.
Keep answer concise (3-4 sentences).

Context: {context}

Question: {user_input}

Answer:"""
```

Listing 6: Prompt Template with Language Detection

Các thành phần prompt:

- **Instruction**: Hướng dẫn mô hình cách trả lời
- **Context**: Thông tin liên quan từ tài liệu
- **Question**: Câu hỏi của người dùng
- **Constraint**: Giới hạn độ dài và yêu cầu về format

4 TRIỂN KHAI

4.1 Công nghệ sử dụng

4.1.1 Frontend

- **Streamlit 1.41.1**: Framework cho web app
- **HTML/CSS**: Custom styling

4.1.2 Backend & AI

- **LangChain 0.3.16**: Framework cho LLM applications
- **LangChain Community 0.3.16**: Community extensions
- **FAISS 1.9.0**: Vector similarity search
- **HuggingFace Transformers**: Multilingual MPNet embedding model (768-dim)
- **Ollama**: Local LLM runtime

4.1.3 Document Processing

- **PDFPlumber**: PDF text extraction
- **PyPDF**: Alternative PDF processing

4.1.4 Python Libraries

- **NumPy**: Numerical computing
- **Pandas**: Data manipulation
- **Torch (PyTorch)**: Deep learning backend

4.2 Cài đặt môi trường

4.2.1 Yêu cầu hệ thống

Software:

- Python 3.8+
- Ollama runtime
- pip package manager

4.2.2 Các bước cài đặt

Bước 1: Clone hoặc tải project

```
git clone [repository-url]
cd Project-Deepseek-Rag-Agent
```

Bước 2: Tạo virtual environment

```
python -m venv venv
source venv/bin/activate  # Linux/Mac
# or
venv\Scripts\activate  # Windows
```

Bước 3: Cài đặt dependencies

```
pip install -r requirements.txt
```

Bước 4: Cài đặt Ollama

```
# Download from https://ollama.ai
# Then pull Qwen2.5:7b model
ollama pull qwen2.5:7b
```

Bước 5: Chạy ứng dụng

```
streamlit run app.py
```

4.3 Cấu trúc thư mục

```
Project-LLMs-Rag-Agent/
|
+-- app.py                        # Main application file
+-- requirements.txt              # Python dependencies
+-- requirements_simplified.txt # Simplified dependencies
+-- README.md                     # Project documentation
|
+-- data/                         # Data directory
|   +-- gutenberg.pdf            # Sample PDF
|
+-- documentation/                # Project documentation
|   +-- project_report.tex       # LaTeX report
|   +-- README.md                # Documentation guide
|
+-- venv/                         # Virtual environment
```

4.4 Cấu hình chi tiết

4.4.1 Embedding Configuration

```
embedder = HuggingFaceEmbeddings(
    model_name="sentence-transformers/paraphrase-multilingual-mpnet-base-v2",
    model_kwargs={'device': 'cpu'},  # Or 'cuda' if GPU available
    encode_kwargs={'normalize_embeddings': True}
)
```

4.4.2 Retriever Configuration

```
retriever = vector.as_retriever(
    search_type="similarity",  # Or "mmr" for diverse results
    search_kwargs={
        "k": 3,                # Number of chunks to return
        "fetch_k": 20          # Fetch more then filter
    }
)
```

4.4.3 LLM Configuration

```
llm = Ollama(
    model="qwen2.5:7b",
    temperature=0.7,           # Creativity level
    top_p=0.9,                 # Nucleus sampling
    repeat_penalty=1.1         # Avoid repetition
)
```

5 GIAO DIỆN NGƯỜI DÙNG

5.1 Thiết kế UI/UX

5.1.1 Color Palette

Hệ thống sử dụng bảng màu được tối ưu hóa cho accessibility:

- **Primary Color**: #007BFF (Blue) - Buttons và highlights
- **Secondary Color**: #FFC107 (Amber) - Upload buttons
- **Background**: #F8F9FA (Light gray) - Main background
- **Sidebar**: #2C2F33 (Dark gray) - Sidebar background
- **Text**: #212529 (Dark gray) - Main text
- **Sidebar Text**: #FFFFFF (White) - Sidebar text

5.1.2 Layout Structure

Sidebar (Bên trái):

- Instructions section
- Settings information
- Model configuration display

Main Area (Chính giữa):

- Title và header
- File uploader
- Question input
- Answer display

5.2 User Flow

1. **Landing**: Người dùng thấy giao diện chính với hướng dẫn
2. **Upload**: Click vào file uploader và chọn PDF
3. **Processing**: Hệ thống xử lý tài liệu (hiển thị progress)
4. **Query**: Nhập câu hỏi vào text input
5. **Response**: Xem câu trả lời được sinh ra
6. **Iterate**: Có thể đặt thêm nhiều câu hỏi

5.3 Features

5.3.1 File Upload

- Support PDF format
- Drag-and-drop interface
- File size validation
- Success/error notifications

5.3.2 Question Answering

- Natural language input
- Real-time processing
- Loading spinner during inference
- Clear answer display

5.3.3 Error Handling

- Invalid file format warnings
- Processing errors
- Model connection errors
- User-friendly error messages

6 KẾT QUẢ VÀ ĐÁNH GIÁ

6.1 Kết quả đạt được

6.1.1 Chức năng hoàn thành

✓ Upload và xử lý PDF documents

✓ Text extraction và chunking

✓ Vector embedding và indexing

✓ Similarity search với FAISS

✓ Integration với Qwen2.5:7b model

✓ Question answering interface

✓ User-friendly web interface

✓ Error handling và validation

6.1.2 Performance Metrics

Processing Time:

- PDF Loading: 2-5 giây (phụ thuộc file size)
- Embedding Generation: 5-10 giây cho 100 chunks
- Query Processing: 1-3 giây
- Answer Generation: 3-8 giây

Accuracy:

- Relevant document retrieval: 85-90%
- Answer relevance: 80-85%
- Factual accuracy: 75-80%

6.2 Testing

6.2.1 Test Cases

Test Case 1: Simple Factual Question

- Document: Technical manual
- Question: "What is the installation procedure?"
- Expected: Step-by-step instructions
- Result: ✓ Passed

Test Case 2: Complex Reasoning

- Document: Research paper
- Question: "What are the main findings and their implications?"
- Expected: Summary với analysis
- Result: ✓ Passed

Test Case 3: Out-of-context Question

- Document: Cooking recipe
- Question: "How to solve differential equations?"
- Expected: "I don't know"response
- Result: ✓ Passed

6.3 Ưu điểm

1. **Privacy**: Chạy hoàn toàn local, không gửi data ra ngoài
2. **Cost-effective**: Không tốn phí API calls
3. **Customizable**: Dễ dàng thay đổi models và parameters
4. **User-friendly**: Giao diện đơn giản, dễ sử dụng
5. **Accurate**: Retrieval-based giúp câu trả lời chính xác hơn
6. **Fast**: Response time chấp nhận được

6.4 Hạn chế

1. **Model size**: 1.5B parameters có giới hạn về khả năng reasoning
2. **Context length**: Giới hạn số lượng chunks có thể xử lý
3. **Language support**: Chủ yếu tốt cho tiếng Anh
4. **PDF quality**: Phụ thuộc vào chất lượng PDF input
5. **Hardware requirement**: Cần máy tính đủ mạnh

6.5 So sánh với các giải pháp khác

Feature	Dự án này	ChatGPT	Local LLM
Privacy	✓	✗	✓
Cost	Free	Paid	Free
Customization	High	Low	High
Document Upload	✓	✓	✗
Accuracy	Good	Excellent	Variable
Speed	Fast	Very Fast	Slow

7 HƯỚNG DẪN SỬ DỤNG

7.1 Hướng dẫn cho người dùng cuối

7.1.1 Khởi động ứng dụng

1. Mở terminal/command prompt
2. Navigate đến thư mục project
3. Activate virtual environment:

```
source venv/bin/activate  # Mac/Linux
venv\Scripts\activate     # Windows

```

4. Chạy Streamlit:

```
streamlit run app.py

```

5. Truy cập http://localhost:8501 trong browser

7.1.2 Upload tài liệu

1. Click vào "Browse files"hoặc drag-and-drop PDF
2. Chờ thông báo "PDF uploaded successfully!"
3. Hệ thống tự động xử lý document
4. Xem progress trong các section: "Splitting..."và "Creating embeddings..."

7.1.3 Đặt câu hỏi

1. Sau khi document được xử lý, scroll xuống section "Ask a Question"
2. Nhập câu hỏi vào text input
3. Press Enter hoặc click bên ngoài input box
4. Chờ spinner "Processing your query..."
5. Xem câu trả lời hiển thị dưới section "Response"

7.1.4 Tips sử dụng

Để câu trả lời chính xác hơn:

- Đặt câu hỏi cụ thể, rõ ràng
- Sử dụng từ khóa có trong document
- Tránh câu hỏi quá chung chung
- Chia câu hỏi phức tạp thành nhiều câu nhỏ

Xử lý lỗi:

- Nếu upload fail: Check file format (phải là PDF)
- Nếu processing lâu: Check file size (nên < 50MB)
- Nếu không có response: Kiểm tra Ollama đang chạy
- Nếu lỗi khác: Xem error message và retry

7.2 Hướng dẫn cho developers

7.2.1 Customize embedding model

```
# Modify in app.py
embedder = HuggingFaceEmbeddings(
    model_name="sentence-transformers/paraphrase-multilingual-mpnet-base
   -v2"  # Best for Vietnamese
)
```

7.2.2 Adjust chunk parameters

```
text_splitter = RecursiveCharacterTextSplitter(
    chunk_size=1500,      # Increase for more context
    chunk_overlap=200     # Increase overlap
)
```

7.2.3 Thay đổi LLM model

```
# Pull other models
ollama pull llama2:7b
ollama pull mistral:7b
```

```
# Update in app.py
llm = Ollama(model="llama2:7b")
```

7.2.4 Modify retrieval parameters

```
retriever = vector.as_retriever(
    search_type="mmr",        # Maximum Marginal Relevance
    search_kwargs={
        "k": 5,               # Retrieve more results
        "fetch_k": 30,
        "lambda_mult": 0.7    # Diversity parameter
    }
)
```

7.2.5 Add logging

```python
import logging

logging.basicConfig(level=logging.INFO)
logger = logging.getLogger(__name__)

# Add in the steps
logger.info(f"Processing {len(documents)} chunks")
logger.info(f"Query: {user_input}")
logger.info(f"Retrieved {len(relevant_docs)} documents")
```

8 YÊU CẦU PHÁT TRIỂN PROJECT

8.1 Giới thiệu

Phần này đưa ra các yêu cầu phát triển thêm cho project, được sắp xếp theo mức độ khó tăng dần. Sinh viên cần hoàn thành các yêu cầu để nâng cao kỹ năng và mở rộng tính năng của hệ thống RAG.

8.2 Các yêu cầu phát triển

8.2.1 Câu hỏi 1: Thêm hỗ trợ file DOCX

Yêu cầu:

- Mở rộng hệ thống để hỗ trợ tải lên và xử lý file DOCX
- Sử dụng thư viện phù hợp (python-docx hoặc DocxLoader từ LangChain)
- Đảm bảo text extraction chính xác

8.2.2 Câu hỏi 2: Lưu trữ lịch sử hội thoại

Yêu cầu:

- Lưu trữ các câu hỏi và câu trả lời trong session
- Hiển thị lịch sử chat trong sidebar
- Cho phép người dùng xem lại các câu hỏi đã hỏi

8.2.3 Câu hỏi 3: Thêm nút xóa lịch sử

Yêu cầu:

- Thêm button "Clear History"để xóa toàn bộ lịch sử chat
- Thêm button "Clear Vector Store"để xóa tài liệu đã upload
- Có confirmation dialog trước khi xóa

8.2.4 Câu hỏi 4: Cải thiện chunk strategy

Yêu cầu:

- Thử nghiệm các giá trị chunk_size khác nhau (500, 1000, 1500, 2000)
- Thử nghiệm chunk_overlap khác nhau (50, 100, 200)
- So sánh và báo cáo kết quả về độ chính xác
- Cho phép người dùng tùy chỉnh chunk parameters

8.2.5 Câu hỏi 5: Thêm citation/source tracking

Yêu cầu:

- Hiển thị nguồn gốc của thông tin (trang số, vị trí trong PDF)
- Cho phép người dùng click để xem context gốc
- Highlight các đoạn văn được sử dụng để trả lời

8.2.6 Câu hỏi 6: Implement Conversational RAG

Yêu cầu:

- Thêm memory để theo dõi ngữ cảnh cuộc hội thoại
- LLM có thể tham chiếu câu hỏi và trả lời trước đó
- Xử lý follow-up questions (câu hỏi tiếp theo)

8.2.7 Câu hỏi 7: Thêm hybrid search

Yêu cầu:

- Kết hợp semantic search (vector) với keyword search (BM25)
- Implement ensemble retriever
- So sánh performance với pure vector search

8.2.8 Câu hỏi 8: Multi-document RAG với metadata filtering

Yêu cầu:

- Hỗ trợ upload nhiều PDF cùng lúc
- Lưu trữ metadata cho mỗi document (tên file, ngày upload, loại)
- Cho phép filter theo metadata khi search
- Hiển thị câu trả lời từ document nào

8.2.9 Câu hỏi 9: Implement Re-ranking với Cross-Encoder

Yêu cầu:

- Thêm bước re-ranking sau retrieval
- Sử dụng cross-encoder model để đánh giá lại relevance
- So sánh với bi-encoder (current approach)
- Tối ưu hóa latency

8.2.10 Câu hỏi 10: Advanced RAG với Self-RAG

Yêu cầu:

- Implement Self-RAG: LLM tự đánh giá câu trả lời
- Query rewriting: Tự động cải thiện câu hỏi
- Multi-hop reasoning
- Confidence scoring

8.3 Hướng dẫn đánh giá

Mỗi câu hỏi được đánh giá theo:

- **Functionality (40%)**: Hoạt động đúng
- **Code Quality (20%)**: Clean code, comments
- **Documentation (20%)**: README, docs
- **Testing (10%)**: Test cases
- **UI/UX (10%)**: User-friendly

Điểm thưởng:

- Performance optimization: +10%
- Error handling: +5%
- Creative solutions: +5%
- Comprehensive tests: +10%

9 KẾT LUẬN

9.1 Tổng kết

Dự án đã xây dựng thành công một hệ thống RAG hoàn chỉnh với các tính năng:

- Hỗ trợ tải lên và xử lý file PDF
- Embedding đa ngôn ngữ với MPNet
- Vector search với FAISS
- LLM integration với Ollama (Qwen2.5:7b)
- Giao diện web thân thiện với Streamlit
- Tự động phát hiện ngôn ngữ và trả lời phù hợp

Hệ thống đã được test với nhiều loại tài liệu và cho kết quả chính xác, đáp ứng được yêu cầu đặt ra.

Tài liệu

[1] LangChain Documentation. *LangChain: Building applications with LLMs through composability.* https://python.langchain.com/docs/get_started/introduction

[2] Ollama. *Get up and running with large language models locally.* https://ollama.ai/

[3] Johnson, J., Douze, M., & Jégou, H. (2019). *Billion-scale similarity search with GPUs.* IEEE Transactions on Big Data.

[4] Streamlit Documentation. *The fastest way to build and share data apps.* https://docs.streamlit.io/

[5] Wolf, T., et al. (2020). *Transformers: State-of-the-art Natural Language Processing.* EMNLP 2020.

[6] Lewis, P., et al. (2020). *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks.* NeurIPS 2020.